

TIER 1 — Absolute Must (Highest ROI, Asked Everywhere)

1. Two Pointers

Used in: arrays, strings, linked lists

- Classic problems: 3Sum, Container With Most Water, Trapping Rain Water, Palindrome Check
- Why: Fast $O(n)$ solutions replacing $O(n^2)$ brute force

2. Sliding Window

- Fixed & variable window
- Classic: Longest Substring Without Repeating, Minimum Window Substring, Max Sum Subarray of Size K
- Why: Meta, Google, Amazon ask this in almost every round

3. Binary Search (beyond textbook)

- On answer space (not just sorted arrays)
- Classic: Search in Rotated Array, Koko Eating Bananas, Median of Two Sorted Arrays
- Why: Google specifically loves "binary search on answer" variants

4. HashMap / HashSet Patterns

- Frequency counting, complement lookup, sliding window combos
 - Classic: Two Sum, Group Anagrams, Subarray Sum Equals K
 - Why: Fastest way to drop time complexity one level
-

TIER 2 — Core Patterns (Company Favorites)

5. Tree Traversals (BFS + DFS — ALL variants)

- Inorder, Preorder, Postorder (iterative + recursive)
- Level-order BFS, Zigzag, Vertical Order, Right Side View
- LCA (Lowest Common Ancestor), Diameter, Max Path Sum
- **Meta is extremely tree-heavy** — dedicate a full week here

6. Graph Algorithms

- BFS (shortest path in unweighted), DFS (connected components, cycle detection)

- **Dijkstra's** (weighted shortest path — Google's favorite follow-up)
- **Union-Find / DSU** (dynamic connectivity)
- Topological Sort (Kahn's Algorithm for dependency problems)
- Classic: Number of Islands, Course Schedule, Clone Graph, Word Ladder

7. Dynamic Programming

The #1 differentiator for senior/marquee roles. Master these patterns:

- **1D DP**: Climbing Stairs, House Robber, Fibonacci
- **2D DP**: Longest Common Subsequence, Edit Distance, Unique Paths
- **Knapsack variants**: 0/1 Knapsack, Unbounded, Partition Equal Subset Sum
- **DP on strings**: Palindromic Substrings, Wildcard Matching
- **Interval DP**: Matrix Chain, Burst Balloons
- **DP + Bitmask**: Traveling Salesman, subset problems

8. Backtracking

- Permutations, Combinations, Subsets (with/without duplicates)
- N-Queens, Sudoku Solver, Word Search
- Key skill: **pruning** to optimize



TIER 3 — Advanced (Separates Good from Great)

9. Heap / Priority Queue

- Top K Elements, K Closest Points, Merge K Sorted Lists
- **Median of Data Stream** (two heaps trick — asked at Google/Meta)
- Sliding Window Maximum (Monotonic Deque variant)

10. Monotonic Stack

- Next Greater Element, Daily Temperatures, Largest Rectangle in Histogram
- **Trapping Rain Water** (Google's #7 most asked)

11. Prefix Sum / Difference Array

- Subarray Sum Equals K, Range Sum Query, Product of Array Except Self

12. Trie (Prefix Tree)

- Implement Trie, Word Search II, AutoComplete
- Amazon and Google both test this for string-heavy roles

13. Greedy Algorithms

- Interval Scheduling (merge intervals, meeting rooms), Jump Game
- Task Scheduler, Minimum Platforms

14. Bit Manipulation

- $n \& (n-1)$ to remove lowest set bit, XOR tricks, counting set bits
 - Single Number, Power of Two, Subsets using bitmask
-



TIER 4 — For L5+/Principal Roles (Super Dream)

15. Segment Tree / BIT (Fenwick Tree)

- Range sum/min/max queries with point updates
- LeetCode 307 (Range Sum Query Mutable)

16. Shortest Path Variants

- **Bellman-Ford** (negative weights), **Floyd-Warshall** (all-pairs)
- **A*** (heuristic-based, system design overlap)

17. LRU / LFU Cache (Design + Algorithm)

- Doubly linked list + HashMap — most asked design-algo combo (tied for #1 across 7 companies)

18. KMP / Rabin-Karp (String Matching)

- For Google/Uber who ask pattern matching problems
-



Company-Specific Focus

Company	Top Priority
Google	Binary Search on answer, Dijkstra, Hard DP, BFS/DFS
Meta	Trees (all traversals), Strings, Two Pointers
Amazon	Arrays, DP, Trees, Greedy, Design (LRU)
Microsoft	Trees, Graphs, DP, Sliding Window
Apple	Clean code + edge cases over exotic algorithms



Suggested Study Order

1. Arrays + HashMap + Two Pointers + Sliding Window (Week 1–2)
2. Binary Search + Prefix Sum + Sorting (Week 3)
3. Trees (all traversals) + Heaps + Stack (Week 4–5)
4. Graphs (BFS/DFS/Dijkstra/Union-Find/Topo Sort) (Week 6–7)
5. Dynamic Programming (1D → 2D → Advanced) (Week 8–10)
6. Backtracking + Greedy + Tries + Bit Manipulation (Week 11–12)
7. Advanced (Segment Tree, LRU/LFU, String algorithms) (Week 13–14)

80/20 Rule: Two Pointers, DP, Binary Search, and HashMaps alone cover ~70% of all asked problems across companies. Master those first before going deeper.

Sources:

- [10 Top LeetCode Patterns to Crack FAANG Coding Interviews](#)
- [Most Asked LeetCode Problems in 2026 — Real Data from 40 Companies](#)
- [FAANG Coding Interview Guide 2026: Google, Meta, Amazon, Netflix & Apple Compared](#)
- [Top 10 DSA Patterns Engineers Must Learn in 2026](#)
- [Complete DSA Roadmap for Product-Based Companies](#)
- [20 DSA Patterns — AlgoMaster](#)
- [Google DS & Algo Interview Preparation Roadmap 2025](#)

can I go with this check and say nanba